

CO4-Assessment Tool-02

CSA0613-Design & Analysis of Algorithms

Name: A.Bhavya(192424417)

Q3. Floyd–Warshall – Network Routing

A telecom company needs to compute shortest paths between all pairs of routers in a network. Analyze how the Floyd–Warshall algorithm can be applied to optimize routing. Design a solution by explaining how intermediate nodes improve shortest path computation. Suggest suitable tools or technologies for implementation. Analyze the time and space complexity of the algorithm. Interpret the results in terms of network efficiency and real-world applicability.

AIM

To apply the Floyd–Warshall algorithm to find the shortest paths between all pairs of routers in a telecom network and analyze its efficiency in network routing.

OBJECTIVE

- To find the shortest path between every pair of routers.
- To understand the role of intermediate routers in routing.
- To analyze the time and space complexity of the algorithm.
- To study its real-world applications in network routing.

INTRODUCTION

In a telecom network, routers are connected through communication links. Each link has a cost that may represent distance, transmission delay, or communication cost.

The Floyd–Warshall algorithm is used to find the shortest paths between all pairs of routers. It is based on dynamic programming and checks whether using an intermediate router can provide a shorter path.

The main formula used is:

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$

Here:

- i represents the source router.
- j represents the destination router.
- k represents an intermediate router.

NETWORK REPRESENTATION

Consider four routers:

A, B, C and D

The initial cost matrix is:

Router	A	B	C	D
A	0	5	∞	10
B	∞	0	3	∞
C	∞	∞	0	1
D	∞	∞	∞	0

Here, ∞ means that there is no direct connection between the routers.

ROLE OF INTERMEDIATE NODES

The Floyd–Warshall algorithm checks every router as an intermediate node.

For example:

The direct path from A to D has a cost of 10.

However, using intermediate routers:

$A \rightarrow B \rightarrow C \rightarrow D$

The total cost is:

$$5 + 3 + 1 = 9$$

Since 9 is less than 10, the algorithm updates the shortest path.

Thus, intermediate nodes help identify better routes and reduce communication costs.

ALGORITHM

- Start the algorithm.
- Input the network graph as a cost matrix.
- Set the initial distance matrix equal to the cost matrix.
- Represent the absence of a direct connection between two routers as ∞ (Infinity).
- Consider each router k as an intermediate router.
- For every source router i and destination router j, check whether the path through router k is shorter.
- Calculate:
 - $\text{dist}[i][k] + \text{dist}[k][j]$
- Compare this value with the current distance:
 - $\text{dist}[i][j]$
- If the new path is shorter, update:
 - $\text{dist}[i][j] = \text{dist}[i][k] + \text{dist}[k][j]$
- Repeat the process for all routers as intermediate nodes.
- After all iterations, the distance matrix contains the shortest paths between all pairs of routers.

- Display the final shortest-path matrix.
- Stop.

PYTHON PROGRAM

```

main.py Visualize Run
1  INF = float('inf')
2
3  routers = ['A', 'B', 'C', 'D']
4
5  graph = [
6      [0, 5, INF, 10],
7      [INF, 0, 3, INF],
8      [INF, INF, 0, 1],
9      [INF, INF, INF, 0]
10 ]
11
12 n = len(graph)
13
14 dist = [row[:] for row in graph]
15
16 for k in range(n):
17     for i in range(n):
18         for j in range(n):
19             if dist[i][k] != INF and dist[k][j] != INF:
20                 new_distance = dist[i][k] + dist[k][j]
21
22                 if new_distance < dist[i][j]:

```

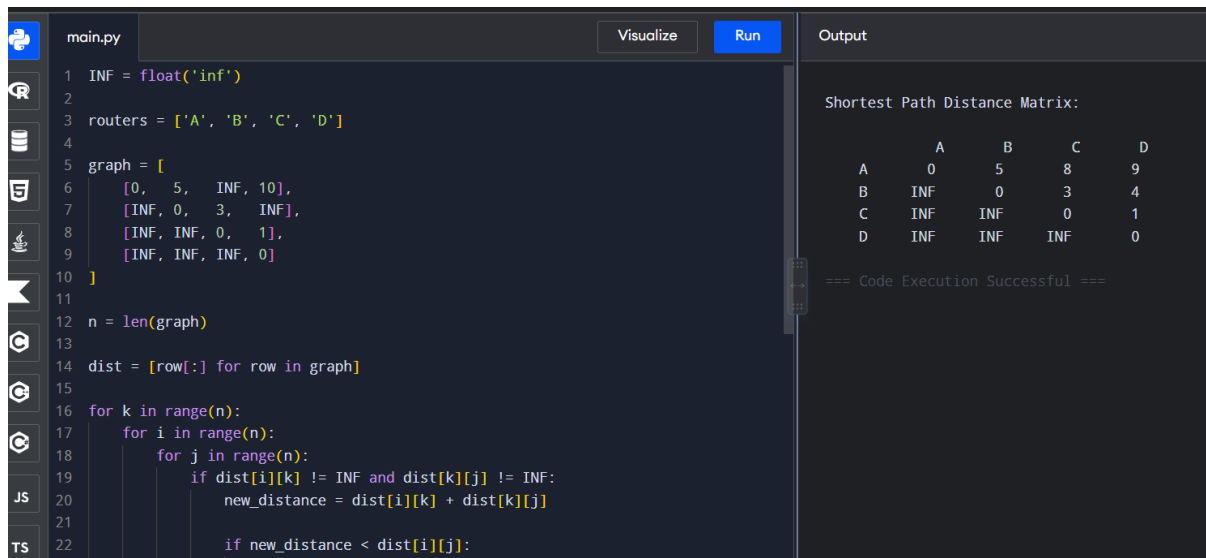
```

main.py Visualize Run
16 for k in range(n):
17     for i in range(n):
18         for j in range(n):
19             if dist[i][k] != INF and dist[k][j] != INF:
20                 new_distance = dist[i][k] + dist[k][j]
21
22                 if new_distance < dist[i][j]:
23                     dist[i][j] = new_distance
24
25 # Display the shortest path matrix
26 print("\nShortest Path Distance Matrix:\n")
27
28 print("      ", end="")
29 for router in routers:
30     print(f"{router:>8}", end="")
31 print()
32
33 for i in range(n):
34     print(f"{routers[i]:>5}", end="")
35
36     for j in range(n):
37         if dist[i][j] == INF:
38             print(f"'INF':>8", end="")
39         else:
40             print(f"{dist[i][j]:>8}", end="")
41
42 print()

```

OUTPUT

Shortest Path Matrix:



```
main.py Visualize Run
1 INF = float('inf')
2
3 routers = ['A', 'B', 'C', 'D']
4
5 graph = [
6     [0, 5, INF, 10],
7     [INF, 0, 3, INF],
8     [INF, INF, 0, 1],
9     [INF, INF, INF, 0]
10 ]
11
12 n = len(graph)
13
14 dist = [row[:] for row in graph]
15
16 for k in range(n):
17     for i in range(n):
18         for j in range(n):
19             if dist[i][k] != INF and dist[k][j] != INF:
20                 new_distance = dist[i][k] + dist[k][j]
21
22                 if new_distance < dist[i][j]:
```

Shortest Path Distance Matrix:

	A	B	C	D
A	0	5	8	9
B	INF	0	3	4
C	INF	INF	0	1
D	INF	INF	INF	0

=== Code Execution Successful ===

[0, 5, 8, 9]

[inf, 0, 3, 4]

[inf, inf, 0, 1]

[inf, inf, inf, 0]

RESULT ANALYSIS

The algorithm finds the minimum communication cost between every pair of routers.

For example:

- $A \rightarrow C$ becomes 8 using $A \rightarrow B \rightarrow C$.
- $A \rightarrow D$ becomes 9 using $A \rightarrow B \rightarrow C \rightarrow D$.
- $B \rightarrow D$ becomes 4 using $B \rightarrow C \rightarrow D$.

This shows that indirect routes can sometimes be more efficient than direct routes.

TOOLS AND TECHNOLOGIES

The following tools can be used:

- Python – For implementing the algorithm.
- Visual Studio Code – For writing and running the program.
- NetworkX – For graph and network analysis.
- Matplotlib – For network visualization.
- Cisco Packet Tracer or GNS3 – For network simulation.

TIME COMPLEXITY

The algorithm uses three nested loops.

Therefore, the time complexity is:

$$O(V^3)$$

where V is the number of routers.

SPACE COMPLEXITY

The algorithm stores the distances between all pairs of routers.

Therefore, the space complexity is:

$$O(V^2)$$

NETWORK EFFICIENCY

The Floyd–Warshall algorithm improves network efficiency by:

- Finding minimum-cost routes.
- Reducing communication delays.
- Using intermediate routers effectively.
- Identifying paths even when no direct connection exists.

- Providing shortest-path information for all routers.

REAL-WORLD APPLICATIONS

The Floyd–Warshall algorithm can be applied in:

- Telecom networks.
- Computer networks.
- Internet routing analysis.
- Transportation systems.
- Data center networks.
- Network planning and optimization.

CONCLUSION

The Floyd–Warshall algorithm is useful for finding the shortest paths between all pairs of routers in a network. By considering intermediate routers, it can discover routes with lower communication costs.

Although the algorithm has a time complexity of $O(V^3)$, it is suitable for small and medium-sized networks where complete routing information is required. The results help improve network efficiency, reduce communication costs, and optimize routing decisions.

RESULT

Thus, the Floyd–Warshall algorithm was successfully applied to compute the shortest paths between all pairs of routers and demonstrate how intermediate nodes improve network routing efficiency.